Name :- Divya Ajay Pandey
Prn :- 202501050016
3.1 Practice Lab Assignments
3.1.1. Numpy Array Operations
3.2 Lab Assignments
3.2.1. NumPy: Matrix Operations
3.2.2. NumPy Horizontal and Vertical Stacking of Arrays
3.2.3. NumPy: Custom Sequence Generation
3.2.4. NumPy: Arithmetic and Statistical Operations,
Mathematical Operations, Bitwise Operators
3.2.5. NumPy: Copying and Viewing Array
3.2.6. NumPy: Searching, Sorting, Counting, Broadcasting
3.2.7. Student Data Analysis and Operations
Practical 03
3.1 Practice Lab Assignments
3.1.1. Numpy Array Operations
Write a Python program to create a NumPy array based on user input and
display the following:
● The created NumPy array.
● The number of dimensions of the array using ndim
● The shape of the array using shape
● The total number of elements in the array using size
Assume all input elements are valid integers.
Input Format:
● The first line contains two space-separated integers,r and c,
representing the number of rows and the number of columns.
● The next r lines contain c space-separated integers representing
the elements of the array, entered row by row.
Output Format:
● The created NumPy array (printed as a 2D array).
● An integer representing the number of dimensions of the array.
● A tuple representing the shape of the array.
● An integer representing the total number of elements in the array.
Note: Use reshape() function to reshape the input array with the
specified number of rows and columns.

```
import numpy as np
# Read the number of rows and columns
r, c = map(int, input().split())
# Read the array elements row by row and flatten them into a single list
elements = []
for _ in range(r):
row = list(map(int, input().split()))
elements.extend(row)
# Create a NumPy array from the flattened list
arr = np.array(elements)
# Reshape the array to the specified dimensions (r rows, c columns)
reshaped_arr = arr.reshape(r, c)
# Display the required outputs
```

```
print(reshaped_arr)
print(reshaped_arr.ndim) #Returns the number of dimensions (2 for a
matrix)
print(reshaped_arr.shape) #Returns a tuple representing the dimensions
(rows, columns)
print(reshaped_arr.size) #Returns the total count of elements in the
```

array

07:20

mitaoe.codetantra.com/

CODETANTRA

```
(rows and columns) from the user
rows, cols = map(int, input().split())

# 2. Collect elements row-wise
elements = []
for _ in range(rows):
    elements.extend(map(int, input().split()))
```

Test case 1

84 ms

Expected output	Actual output
3 4	3 4
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
9 10 11 12	9 10 11 12
[[1 2 3 4]	[[1 2 3 4]

07:20

mitaoe.codetantra.com/

CODETANTRA

```
(rows and columns) from the user
rows, cols = map(int, input().split())

# 2. Collect elements row-wise
elements = []
for _ in range(rows):
    elements.extend(map(int, input().split()))
```

[[1 2 3 4]	[[1 2 3 4]
[5 6 7 8]	[5 6 7 8]
[9 10 11 12]]	[9 10 11 12]]
2	2
(3, 4)	(3, 4)
12	12

3.2 Lab Assignments

3.2.1. NumPy: Matrix Operations

The given code takes two 3×3 matrices, matrix_a, and matrix_b, as input from the user and converts them into NumPy arrays.

Task:You are required to compute and display the results of the following matrix operations:

1. Addition (matrix_a + matrix_b)
2. Subtraction (matrix_a - matrix_b)
3. Element-wise Multiplication (matrix_a * matrix_b)
4. Matrix Multiplication (matrix_a • matrix_b)
5. Transpose of Matrix A

Input Format:

● The user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.

● Similarly, the user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

```
import numpy as np
# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
# Addition
print("Addition (A + B):")
a = matrix_a + matrix_b
print(a)
# Subtraction
print("Subtraction (A - B):")
b = matrix_a - matrix_b
print(b)
# Multiplication (element-wise)
print("Element-wise Multiplication (A * B):")
m = matrix_a*matrix_b
print(m)
# Matrix multiplication (dot product)
print("A dot B:")
d = np.dot(matrix_a,matrix_b)
print(d)
# Transpose
print("Transpose of A:")
```

```
t = np.transpose(matrix_a)
```

```
print(t)
```

07:22

mitaoe.codetantra.com/

CODETANTRA

```
print("Subtraction (A - B):")
print(subtraction_result)
# Multiplication (element-wise)
elementwise_multiplication_result = matrix_a * matrix_b
print("Element-wise Multiplication (A * B):")
print(elementwise_multiplication_result)
# Matrix multiplication
```

Test case 1

133 ms

Expected output	Actual output
Enter Matrix A:	Enter Matrix A:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9

07:22

mitaoe.codetantra.com/

CODETANTRA

```
print("Subtraction (A - B):")
print(subtraction_result)
# Multiplication (element-wise)
elementwise_multiplication_result = matrix_a * matrix_b
print("Element-wise Multiplication (A * B):")
print(elementwise_multiplication_result)
# Matrix multiplication
```

```
Enter Matrix B:
1 1 1
1 1 1
1 1 1
Addition (A + B):
[[ 2  3  4]
 [ 5  6  7]
```

```
1 1 1
1 1 1
1 1 1
Addition (A + B):
[[ 2  3  4]
 [ 5  6  7]
```

07:22

mitaoe.codetantra.com/

CODETANTRA

```
print("Subtraction (A - B):")
print(subtraction_result)
# Multiplication (element-wise)
elementwise_multiplication_result = matrix_a * matrix_b
print("Element-wise Multiplication (A * B):")
print(elementwise_multiplication_result)
# Matrix multiplication
```

```
 [ 8  9 10]]
Subtraction (A - B):
[[0 1 2]
 [3 4 5]
 [6 7 8]]
Element-wise Multiplication (A *
```

```
 [ 8  9 10]]
Subtraction (A - B):
[[0 1 2]
 [3 4 5]
 [6 7 8]]
Element-wise Multiplication (A *
```

07:22

mitaoe.codetantra.com/

CODETANTRA

```
print("Subtraction (A - B):")
print(subtraction_result)
# Multiplication (element-wise)
elementwise_multiplication_result = matrix_a * matrix_b
print("Element-wise Multiplication (A * B):")
print(elementwise_multiplication_result)
# Matrix multiplication
```

```
A dot B:
[[ 6  6  6]
 [15 15 15]
 [24 24 24]]
Transpose of A:
[[1 4 7]
 [2 5 8]
```

```
A dot B:
[[ 6  6  6]
 [15 15 15]
 [24 24 24]]
Transpose of A:
[[1 4 7]
 [2 5 8]
```

3.2.2. NumPy Horizontal and Vertical Stacking of Arrays

You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.

● Horizontal Stacking: Stack the two matrices horizontally (side by side).

● Vertical Stacking: Stack the two matrices vertically (one below the other).

Input Format:

● The program should first prompt the user to input two 3x3 arrays.

● Each array consists of 3 rows, and each row contains 3 space-separated integers.

● The user will input the two arrays row by row.

Output Format:

● The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.

● The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

```
import numpy as np
# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])
print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])
# Perform horizontal stacking (hstack)
h_stack = np.hstack((arr1, arr2))
# Perform vertical stacking (vstack)
v_stack = np.vstack((arr1, arr2))
# Display the results
print("Horizontal Stack:")
print(h_stack)
print("Vertical Stack:")
print(v_stack)
```

07:24

mitaoe.codetantra.com/

CODETANTRA

```
in range(3)])

# Perform horizontal stacking (hstack)
a = np.hstack((arr1, arr2))
print("Horizontal Stack:")
print(a)

# Vertical Stacking
b = np.vstack((arr1, arr2))
```

Test case 1

80 ms

Expected output	Actual output
Enter Array1:	Enter Array1:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Array2:	Enter Array2:

07:24

mitaoe.codetantra.com/

CODETANTRA

```
in range(3)])

# Perform horizontal stacking (hstack)
a = np.hstack((arr1, arr2))
print("Horizontal Stack:")
print(a)

# Vertical Stacking
b = np.vstack((arr1, arr2))
```

10 11 12	10 11 12
Horizontal Stack:	Horizontal Stac k:
[[1 2 3 4 5 6]	[[1 2 3 4 5 6]
[4 5 6 7 8 9]	[4 5 6 7 8 9]
[7 8 9 10 11	[7 8 9 10 11

07:24

mitaoe.codetantra.com/

CODETANTRA

```
in range(3)])

# Perform horizontal stacking (hstack)
a = np.hstack((arr1, arr2))
print("Horizontal Stack:")
print(a)

# Vertical Stacking
b = np.vstack((arr1, arr2))
```

```
 [ 7  8  9 10 11 12]]
Vertical Stack:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [ 4  5  6]
 [ 7  8  9]
```

```
 [ 7  8  9 10 11 12]]
Vertical Stack:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [ 4  5  6]
 [ 7  8  9]
```

3.2.3. NumPy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

● Start value: The starting point of the sequence.

● Stop value: The sequence should end before this value.

● Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

● The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

● The program should print the generated sequence based on the input values.

```
import numpy as np
# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())
# Generate the sequence using np.arange()
arr = np.arange(start,stop,step)
# Print the generated sequence
print(arr)
```

07:26

mitaoe.codetantra.com/

CODETANTRA

```
start = int(input())
stop = int(input())
step = int(input())

# Generate the sequence using np.arange()

# Print the generated sequence
a = np.arange(start, stop, step)

# Print the generated
```

2 out of 2 hidden test case(s) passed

Test case 1

77 ms

Expected output	Actual output
3	3
10	10
2	2

3.2.4. NumPy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators

You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:
● Compute the element-wise sum, difference, and product of the two arrays.
2. Statistical Operations:
● Calculate the mean, median, and standard deviation of array A.
3. Bitwise Operations:
● Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: Ai OR Bi).

Input Format:
● The first line contains space-separated integers representing the elements of array A.
● The second line contains space-separated integers representing the elements of array B.

Output Format:
● For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

```
import numpy as np
def array_operations(A, B):
# Convert A and B to NumPy arrays
arr_A = np.array(A)
arr_B = np.array(B)
# Arithmetic Operations
sum_result = np.add(arr_A, arr_B)
diff_result = np.subtract(arr_A, arr_B)
prod_result = np.multiply(arr_A, arr_B)
# Statistical Operations
mean_A = np.mean(arr_A)
median_A = np.median(arr_A)
std_dev_A = np.std(arr_A)
# Bitwise Operations
and_result = np.bitwise_and(arr_A, arr_B)
or_result = np.bitwise_or(arr_A, arr_B)
xor_result = np.bitwise_xor(arr_A, arr_B)
# Output results with one space between each element
print("Element-wise Sum:", ' '.join(map(str, sum_result)))
print("Element-wise Difference:", ' '.join(map(str, diff_result)))
print("Element-wise Product:", ' '.join(map(str, prod_result)))
print(f"Mean of A: {mean_A}")
print(f"Median of A: {median_A}")
print(f"Standard Deviation of A: {std_dev_A}")
print("Bitwise AND:", ' '.join(map(str, and_result)))
print("Bitwise OR:", ' '.join(map(str, or_result)))
print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

```
A = list(map(int, input().split())) # Elements of array A
B = list(map(int, input().split())) # Elements of array B
```

array_operations(A, B)

07:27

mitaoe.codetantra.com/

CODETANTRA

```
    mean_A = np.mean(A)
    median_A = np.median(A)
    std_dev_A = np.std(A)

    # Bitwise Operations
    and_result = A & B
    or_result = A | B
    xor_result = A ^ B

    # Output results with one space between each element
```

Test case 1

79 ms

Expected output	Actual output
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
Element-wise Sum: 6 8 10 12	Element-wise Sum: 6 8 10 12
Element-wise Diff	Element-wise Dif

07:27

mitaoe.codetantra.com/

CODETANTRA

```
mean_A = np.mean(A)
median_A = np.median(A)
std_dev_A = np.std(A)

# Bitwise Operations
and_result = A & B
or_result = A | B
xor_result = A ^ B

# Output results with one space between each element
```

uct: 5 12 21 32	duct: 5 12 21 32
Mean of A: 2.5	Mean of A: 2.5
Median of A: 2.5	Median of A: 2.5
Standard Deviation of A: 1.118033988749895	Standard Deviation of A: 1.11803398874989 5
Bitwise AND: 1 2	Bitwise AND: 1 2

07:27

mitaoe.codetantra.com/

CODETANTRA

```
mean_A = np.mean(A)
median_A = np.median(A)
std_dev_A = np.std(A)

# Bitwise Operations
and_result = A & B
or_result = A | B
xor_result = A ^ B

# Output results with one space between each element
```

Median of A: 2.5	
Standard Deviation of A: 1.118033988749895	Standard Deviation of A: 1.118033988749895
Bitwise AND: 1 2 3 0	Bitwise AND: 1 2 3 0
Bitwise OR: 5 6 7 12	Bitwise OR: 5 6 7 12
Bitwise XOR: 4 4	Bitwise XOR: 4 4

3.2.5. NumPy: Copying and Viewing Array

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

● Creating a view of the original_array and assigning it to view_array.

● Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

● A single line of space-separated integers.

Output Format:

● After modifying the view:

Original array after modifying view: <original_array>

View array: <view_array>

● After modifying the copy:

Original array after modifying copy: <original_array>

Copy array: <copy_array>

```
import numpy as np
inputlist = list(map(int,input().split(" ")))
# Original array
original_array = np.array(inputlist)
# Create a view of the original_array
view_array = original_array.view()
# Create a copy of the original_array
copy_array = original_array.copy()
# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)
# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
```

```
print("Copy array:", copy_array)
```

07:28

mitaoe.codetantra.com/

CODETANTRA

```
# Create a view
view_array =original_array.view()

# Create a copy
copy_array =original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
```

Test case 1

49 ms

Expected output	Actual output
10 20 30 40 50 60 70 80	10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]	Original array after modifying view: [99 20 30 40 50 60 70 8

07:28

mitaoe.codetantra.com/

CODETANTRA

```
# Create a view
view_array =original_array.view()

# Create a copy
copy_array =original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
```

```
Original array after modifying copy: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]
```

3.2.6. NumPy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. Searching: Find the indices where search_value appears in array1 and print these indices.
2. Counting: Count how many times count_value appears in array1 and print the count.
3. Broadcasting: Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
4. Sorting: Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

```
import numpy as np
# Input array from the user
array1 = np.array(list(map(int, input().split())))
# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))
# Find indices where value matches in array1
indices = np.where(array1 = search_value)[0]
print(indices)
# Count occurrences in array1
count = np.count_nonzero(array1 = count_value)
print(count)
# Broadcasting addition
broadcasted_array = array1 + broadcast_value
print(broadcasted_array)
# Sort the first array
sorted_array = np.sort(array1)
print(sorted_array)
```

07:29

mitaoe.codetantra.com/

CODETANTRA

```
# 2. Counting: Count occurrences of count_value
# b = np.count_nonzero(array1 == count_value) is also valid
b = np.sum(array1 == count_value)
print(b)

# 3. Broadcasting: Add
```

Test case 1

101 ms

Expected output	Actual output
1 1 1 2 2 2	1 1 1 2 2 2
Value to search: 1	Value to search: 1
Value to count:	Value to count:

07:29

mitaoe.codetantra.com/

CODETANTRA

```
search_value)[0]
print(a)

# 2. Counting: Count occurrences of count_value
# b = np.count_nonzero(array1 == count_value) is also valid
b = np.sum(array1 == count_value)
print(b)

# 3. Broadcasting: Add
```

Explorer

Value to add: 2	Value to add: 2
[0 1 2]	[0 1 2]
3	3
[3 3 3 4 4 4]	[3 3 3 4 4 4]
[1 1 1 2 2 2]	[1 1 1 2 2 2]

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

● Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
● Find total students: Determine the total number of students in the dataset.
● Print all student roll numbers: Extract and print the roll numbers of all students.
● Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
● Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
● Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
● Print all subject marks: Display the marks of all students for each subject.
● Find total marks of students: Compute the total marks for each student across all subjects.
● Find the average marks of each student: Compute the average marks for each student.
● Find average marks of each subject: Compute the average marks for all students in each subject.
● Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
● Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
● Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
● Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
● Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
● Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.

● Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
● Find the count of students who scored >=90 in each subject: Count the number of students who scored 90 or more marks in each subject.
● Find the count of subjects in which each student scored >=90:

Determine how many subjects each student scored 90 or more marks in.

● Print Subject 1 marks in ascending order: Sort and print the marks of students in Subject 1 in ascending order.

● Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.

○ Note: There is a Ghost Table in the Test Case. Print the original array to pass it.

● Find index positions of students who scored 79 in Subject 1: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

```
import numpy as np
a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
# 1. Print all student details
print("All student Details:\n", a)
# 2. Find total students
print("Total Students:", int(a.shape[0]))
# 3. Print all student roll numbers
print("All Student Roll Nos", a[:, 0])
# 4. Print Subject 1 marks
print("Subject 1 Marks", a[:, 1])
# 5. Find minimum marks in Subject 2
print("Min marks in Subject 2", np.min(a[:, 2]))
# 6. Find maximum marks in Subject 3
print("Max marks in Subject 3", np.max(a[:, 3]))
# 7. Print all subject marks
print("All subject marks:", a[:, 1:])
# 8. Find total marks of students
print("Total Marks", np.sum(a[:, 1:], axis=1))
# 9. Find the average marks of each student
# Rounding to 1 decimal place matches the sample output format
print(np.round(np.mean(a[:, 1:], axis=1), 1))
# 10. Find average marks of each subject
print("Average Marks of each subject", np.mean(a[:, 1:], axis=0))
# 11. Find average marks of Subject 1 and Subject 2
print("Average Marks of S1 and S2", np.mean(a[:, 1:3], axis=0))
# 12. Find average marks of Subject 1 and Subject 3
print("Average Marks of S1 and S3", np.mean(a[:, [1, 3]], axis=0))
# 13. Find the roll number of the student with maximum marks in Subject3
max_s3_index = np.argmax(a[:, 3])
print("Roll no who got maximum marks in Subject 3", a[max_s3_index, 0])
# 14. Find the roll number of the student with minimum marks in Subject2
min_s2_index = np.argmin(a[:, 2])
print("Roll no who got minimum marks in Subject 2", a[min_s2_index, 0])
# 15. Find the roll number of students who scored 24 marks in Subject 2
# We use slicing [:, [0]] to maintain the 2D structure as shown in the
```

```
sample output [[310.]]
roll_24 = a[a[:, 2] = 24][:, [0]]
print("Roll no who got 24 marks in Subject 2", roll_24)
# 16. Find the count of students who got less than 40 marks in Subject 1
count_s1_less_40 = np.sum(a[:, 1] < 40)
print("Count of students who got marks in Subject 1 < 40",
count_s1_less_40)
# 17. Find the count of students who got more than 90 marks in Subject 2
count_s2_more_90 = np.sum(a[:, 2] > 90)
print(f"Count of students who got marks in Subject 2 > 90:
{count_s2_more_90}")
# 18. Find the count of students who scored =90 in each subject
count_each_sub_90 = np.sum(a[:, 1:] = 90, axis=0)
print("Count of students in each subject who got marks = 90:",
count_each_sub_90)
# 19. Find the count of subjects in which each student scored =90
# First print Roll numbers as per sample output
print("Roll no:", a[:, 0])
count_stud_sub_90 = np.sum(a[:, 1:] = 90, axis=1)
print("Count of subjects in which student got marks = 90:",
count_stud_sub_90)
# 20. Print Subject 1 marks in ascending order
print(np.sort(a[:, 1]))
# 21. Print students who scored between 50 and 90 in Subject 1
# Filtering rows where Subject 1 (column 1) is = 50 and = 90
students_50_90 = a[(a[:, 1] = 50) & (a[:, 1] = 90)]
print(students_50_90)
#Ghost Table: Print the Original Array to pass it
print(a)
# 22. Find index positions of students who scored 79 in Subject 1
indices_79 = np.where(a[:, 1] = 79)
print(indices_79)
```

07:29

mitaoe.codetantra.com/

CODETANTRA

```
print("Roll no:",a[:,0])
print("Count of subjects in which student got marks >= 90:",np.count_nonzero(a[:,1:]>90, axis=1))

# 20. Print S1 marks in ascending order
srt=np.sort(a[:,1])
print(srt)

# 21. Print S1 marks >=
```

Exp

Test case 1

79 ms

Expected output	Actual output
All student Details:	All student Details:
[[301. 67. 77. 88.]	[[301. 67. 77. 88.]
[302. 78. 88.	[302. 78. 8

07:29

mitaoe.codetantra.com/

CODETANTRA

```
print("Roll no:",a[:,0])
print("Count of subjects in which student got marks >= 90:",np.count_nonzero(a[:,1:]>90, axis=1))

# 20. Print S1 marks in ascending order
srt=np.sort(a[:,1])
print(srt)

# 21. Print S1 marks >=
```

```
[310.  79.  24.  33.]]
Total Students: 10
All Student Roll Nos [301. 302. 303. 304. 305. 306. 307. 308. 309. 310.]
```

```
[310.  79.  24.  33.]]
Total Students: 10
All Student Roll Nos [301. 302. 303. 304. 305. 306. 307. 308. 309. 310.]
```

07:30

mitaoe.codetantra.com/

CODETANTRA

```
print("Roll no:",a[:,0])
print("Count of subjects in which student got marks >= 90:",np.count_nonzero(a[:,1:]>90, axis=1))

# 20. Print S1 marks in ascending order
srt=np.sort(a[:,1])
print(srt)

# 21. Print S1 marks >=
```

```
8.]
[78. 88. 77.]      [78. 88. 77.]
[45. 56. 89.]      [45. 56. 89.]
[88. 98. 45.]      [88. 98. 45.]
[78. 88. 99.]      [78. 88. 99.]
[68. 78. 36.]      [68. 78. 36.]
[79. 12. 45.]      [79. 12. 45.]
[46. 45. 77.]      [46. 45. 77.]
```

07:30

mitaoe.codetantra.com/

CODETANTRA

```
print("Roll no:",a[:,0])
print("Count of subjects in which student got marks >= 90:",np.count_nonzero(a[:,1:]>90, axis=1))

# 20. Print S1 marks in ascending order
srt=np.sort(a[:,1])
print(srt)

# 21. Print S1 marks >=
```

```
[79. 24. 33.]]

Total Marks [232. 243. 190. 231. 265. 182. 136. 168. 209. 136.]

[77.3 81.  63.3 77.  88.3 60.7 45.3 56.  69.7 45.3]
```

```
Total Marks [232. 243. 190. 231. 265. 182. 136. 168. 209. 136.]

[77.3 81.  63.3 77.  88.3 60.7 45.3 56.  69.7 45.3]
```

07:30

mitaoe.codetantra.com/

CODETANTRA

```
print("Roll no:",a[:,0])
print("Count of subjects in which student got marks >= 90:",np.count_nonzero(a[:,1:]>90, axis=1))

# 20. Print S1 marks in ascending order
srt=np.sort(a[:,1])
print(srt)

# 21. Print S1 marks >=
```

```
Roll no who got maximum marks in Subject 3 305.0
Roll no who got minimum marks in Subject 3 305.0
Roll no who got minimum marks in Subject 2 307.0
Roll no who got minimum marks in Subject 2 307.0
Roll no who got 24 marks in Subject 2 [[31
Roll no who got 24 marks in Subject 2 [[31
```

07:30

mitaoe.codetantra.com/

CODETANTRA

```
print("Roll no:",a[:,0])
print("Count of subjects in which student got marks >= 90:",np.count_nonzero(a[:,1:]>90, axis=1))

# 20. Print S1 marks in ascending order
srt=np.sort(a[:,1])
print(srt)

# 21. Print S1 marks >=
```

Exp

```
Count of students who got marks in Subject 2 > 90: 1
Count of students in each subject who got marks >= 90: [0 1 1]
Roll no: [301.
```

```
Count of students who got marks in Subject 2 > 90: 1
Count of students in each subject who got marks >= 90: [0 1 1]
```

07:30

mitaoe.codetantra.com/

CODETANTRA

```
print("Roll no:",a[:,0])
print("Count of subjects in which student got marks >= 90:",np.count_nonzero(a[:,1:]>90, axis=1))

# 20. Print S1 marks in ascending order
srt=np.sort(a[:,1])
print(srt)

# 21. Print S1 marks >=
```

Count of subjects in which student got marks >= 90: [0 0 0 1 1 0 0 0 0 0]	Count of subjects in which student got marks >= 90: [0 0 0 1 1 0 0 0 0 0]
[45. 46. 67. 68. 78. 78. 79. 79. 88. 89.]	[45. 46. 67. 68. 78. 78. 79. 79. 88. 89.]
[[301. 67. 77.	[[301. 67. 7

07:30

mitaoe.codetantra.com/

CODETANTRA

```
print("Roll no:",a[:,0])
print("Count of subjects in which student got marks >= 90:",np.count_nonzero(a[:,1:]>90, axis=1))

# 20. Print S1 marks in ascending order
srt=np.sort(a[:,1])
print(srt)

# 21. Print S1 marks >=
```

```
 45.]                    8.  45.]
[305.  78.  88.          [305.  78.  8
 99.]                    8.  99.]
[306.  68.  78.          [306.  68.  7
 36.]                    8.  36.]
[307.  79.  12.          [307.  79.  1
 45.]                    2.  45.]
[308.  46.  45.          [308.  46.  4
 77.]                    5.  77.]
```